

OMB Portfolio Builder Curated File Guides

Book-style explanations for selected important files. This first curated pass covers `server.mjs`, `script.js`, and `index.html`.

How This Curated Set Is Different

The generated source references are useful as raw maps, but they are not how a beginner learns a system. This curated set starts each file with a long overview, then explains code by responsibility. It avoids line-number references because source lines will move as the application evolves.

The first covered files are `server.mjs`, `script.js`, and `index.html` because they form the local backend, public runtime, and public shell of the portfolio website.

Curated File Guide: server.mjs

The local backend that saves drafts, runs compilers, checks GitHub publishing access, imports/pushes site files, handles updates, and relays AI.

Before The Code: File Overview

Abstract

server.mjs is the local nervous system of the OMB Portfolio Builder. It is not the public website and it is not the Electron shell. It is the private backend that runs on the owner's machine and gives the builder the powers a browser page should not have by itself: reading and writing local files, saving drafts, accepting uploads, running compilers, using Git, checking publishing authorization, importing a target site, launching installers, and relaying AI requests. The file is intentionally broad because the builder is local-first. Instead of depending on a hosted database and a remote application server, the desktop app starts this Node server beside the user-owned workspace and talks to it through local API endpoints.

The most important idea is boundary control. The frontend can draw buttons and collect input, but it cannot safely decide where files are written, which Git repository receives a push, whether a command should run, or whether a request came from the local computer. server.mjs owns those decisions. It is the layer that turns a UI action into a guarded filesystem, compiler, Git, installer, or AI operation.

Introduction

When the builder starts, Electron loads a local page. That page needs catalog data, templates, upload handling, compile actions, publishing actions, and update checks. A static HTML page cannot perform those operations directly without either exposing dangerous privileges or running into browser security walls. server.mjs solves that by becoming a small local application server. It serves the builder files, exposes API endpoints under /api, and keeps privileged work in Node where the filesystem, process runner, and Git commands are available.

A useful mental model is to treat this file like a workshop manager. The frontend is the workbench surface where the user sees projects, editors, buttons, and dialogs. server.mjs is the person behind the counter who knows where the tools are, where files belong, whether a customer has permission to ship something, and whether the requested operation is safe. If the frontend says 'save this PDF under this project,' server.mjs sanitizes the path and writes it under the correct docs folder. If the frontend says 'compile this C++ program,' server.mjs finds g++, creates a temporary run folder, captures terminal output, and returns a result object. If the frontend says 'apply to site,' server.mjs checks Git authorization first, then synchronizes publishable files and pushes them.

Background Information

The file uses built-in Node modules rather than a large web framework. createServer from node:http accepts requests, node:fs/promises handles files, node:child_process runs compilers and Git, node:path normalizes paths, node:crypto hashes authorization cache scopes, and node:os identifies the current machine. This keeps the backend predictable and package-light. The tradeoff is that routing and request handling are written manually. That is why handleApi is large: it plays the role that Express or another framework would normally play.

There are three important roots. root is the builder workspace that contains the site files and local draft catalog. portfolioRoot is the publish mirror, which may be separate from root so generated website files can be pushed without mixing in builder-only files. compileRoot is where project-local code workspaces and temporary compiler run folders live. Most path-related functions exist to make sure a request stays inside one of these roots.

The backend is also intentionally local-sensitive. Some endpoints are read-only, but write endpoints reject non-local callers. That is why functions such as `isLocalRequest` matter: the builder may be opened in a browser or accessed on a LAN during testing, but operations that change files, compile code, install tools, or publish should be performed only from the owner's computer.

Purpose Of The File

The purpose of `server.mjs` is to make the builder operational instead of decorative. Without it, `template-preview.html` and `template-preview.js` would still be able to render a screen, but they would not be able to reliably save drafts, upload project evidence, run compilers, authenticate GitHub targets, import an existing website, launch updates, or call private AI providers. The file therefore holds the application's real side effects.

The file is also a protection layer. It validates filenames, normalizes repository URLs, keeps publish authentication scoped to a machine and target, prevents stageable publishing paths from wandering outside the publish mirror, blocks remote write requests, caps text fetched from external sources, and strips terminal output into UI-safe strings. Those decisions are not cosmetic. They prevent the portfolio builder from becoming a general-purpose remote command runner or a tool that accidentally pushes the wrong files.

Primary Responsibilities

The first responsibility is serving local files. The same server that handles `/api` requests also serves `index.html`, `styles.css`, `script.js`, `template-preview.html`, project catalogs, assets, and uploaded docs during local preview. The static-serving logic checks that the requested path stays inside the workspace before reading the file. That simple check is what prevents a URL from escaping into arbitrary folders on the machine.

The second responsibility is compilation. Compile Code works because the backend can write source files to a project workspace, discover tools such as `node`, `python`, `gcc`, `g++`, `javac`, `iverilog`, `vvp`, and `LTspice`, run those tools with timeouts, collect stdout and stderr, cache successful builds, and return terminal output immediately to the frontend. The backend treats each language differently because JavaScript, Python, C, C++, Java, Verilog, SystemVerilog, LTspice, and HTML do not build the same way.

The third responsibility is publishing. Publishing is not just 'write `projects.json`.' The backend must verify that the publish mirror is a Git repository, know its origin remote and branch, check compatibility with a static portfolio site, synchronize with the remote to avoid non-fast-forward errors, stage only approved publish paths, commit if there are changes, and push to the selected branch. This is why the publishing code is careful and verbose.

The fourth responsibility is intelligence. For local AI, the backend can use an OpenAI API key or a local Ollama model. It also enriches portfolio context by reading public project links, same-site files, GitHub repositories, GitHub profiles, README files, and selected source files when the question calls for it. The browser cannot safely hold private API keys, so `server.mjs` keeps that path private.

The fifth responsibility is application maintenance. The update functions check GitHub Releases, download an installer, write a PowerShell handoff script, close the running app, run the installer, and relaunch the updated executable. This is backend work because the browser cannot replace the running desktop app.

Important Data Objects And Why They Matter

`compileLanguageProfiles` is the backend's language map. It tells the compiler system what default filename to use, which file extensions belong to each language, what human label to show, which tools are required, and which Winget package can install the tool if automatic installation exists. Without this object, `compileAndRunCode` would need hardcoded language assumptions scattered throughout the file.

`compileToolCandidates` is the tool-discovery map. It lists command names and likely Windows install paths for Git-adjacent and compiler-adjacent executables. `findTool` uses this object so the app can work even when a tool

is installed but not perfectly exposed on PATH. The caches `compileToolCache` and `compileToolVersionCache` exist so repeated compiles do not waste time rediscovering the same executables.

`publishPaths` is the publishing allowlist. It lists the site files and folders the backend is allowed to copy into the publish mirror and stage in Git. This is one of the most important safety objects in the file. Without an allowlist, an Apply to site operation could accidentally stage builder-only files, private drafts, temporary compile output, or unrelated local files.

`portfolioAiInstructions` is the system behavior contract for the portfolio assistant. It tells the model how to distinguish general conversation, general engineering, general knowledge, and portfolio-specific questions. It also tells the model when to use context, when not to invent, and how to handle public GitHub source excerpts. That string is long because it is the policy boundary for the AI feature.

Code Walkthrough

The walkthrough below groups related functions where they form one idea. Small helpers are explained together when that is clearer than pretending each one is a separate chapter. Larger functions receive direct explanations of their parameters, internal objects, side effects, return values, and why they are necessary.

Startup, roots, and response safety

These functions and constants establish the backend's working environment. They define where files live, how JSON responses are sent, and what security headers are attached.

securityHeaders(extra)

This function returns the baseline security headers added to local backend responses. The optional extra object lets a caller add or override headers without rewriting the common policy. Its purpose is to keep every response consistent: no MIME sniffing, strict referrer behavior, and browser permission limits. Without it, every endpoint would need to remember its own security headers, and missed endpoints could behave differently.

sendJson(response, status, data)

This is the standard JSON response writer. It receives the Node response object, an HTTP status code, and any serializable data object. It writes headers, disables caching, stringifies the data with indentation, and closes the response. The function matters because every API endpoint should return JSON in the same shape and with the same cache rules. It is intentionally synchronous around `response.end` because once the data object is ready, the output should be immediate.

createServer request handler

The final `createServer` block is the actual HTTP entry point. It first asks `handleApi` whether an `/api` request has been handled. If not, it treats the request as a static file request. The pathname is normalized, joined to root, and rejected if it escapes root. Then the file is read and sent with a MIME type from the types map. This is the simplest possible local server: API first, static files second, forbidden paths rejected, missing files return 404.

Language detection, filenames, and code formatting

These helpers make Compile Code feel automatic. They normalize language names, infer languages from file names and source text, produce safe filenames, and apply simple beautification before code is saved or displayed.

normalizeCodeLanguage(value), languageFromFileName(fileName, code), and detectCodeLanguageFromSource(code, fileName)

These functions form the language-detection chain. `normalizeCodeLanguage` turns aliases such as `js`, `py`, `cpp`, `c++`, `sv`, or `system verilog` into canonical ids. `languageFromFileName` prefers file extensions because `.java`, `.sv`, `.py`, and `.cpp` are strong evidence. `detectCodeLanguageFromSource` uses both filename and source clues, such as `public class` for Java or `module` syntax for HDL. The returned language id selects a compile profile, default filename, toolchain, highlighting rules, and build behavior.

safeCodeFileName(value, language)

This function accepts a requested filename and a language, then returns a filename safe enough to write under the compile workspace. It strips dangerous path characters, keeps a language-appropriate extension, and falls back to the default filename from `compileLanguageProfiles`. Without it, a user-provided filename could accidentally create confusing paths or miss the extension needed by a compiler.

beautifyCode(code, language) and indentBraceCode(code)

`beautifyCode` is a lightweight formatter used by the code editor. For brace-based languages it delegates to `indentBraceCode`, which adjusts indentation by watching braces and brackets. It is not a full compiler-grade

formatter, but it gives pasted code a cleaner shape. The function exists because code that looks organized is easier to read before being appended to a project.

Tool discovery and process execution

Compile Code, Git publishing, installers, and AI helpers all need controlled process execution. These functions find tools, run them safely, and return useful terminal text.

findExecutableUnder(folder, names, maxDepth) and findTool(toolName)

findExecutableUnder recursively searches likely install folders for executable names. findTool uses compileToolCandidates first, then command lookup, then known package folders. The important parameter is toolName: it is the key into the candidate map and cache. The function is async because filesystem searches and command lookup can take time. It stores successful results in compileToolCache so repeated compiler runs do not lag.

runProcess(command, args, options)

runProcess is the controlled command runner. command is the executable path or command name; args is the exact argument list; options can supply cwd, timeoutMs, env, and stdin input. The function spawns the process, collects stdout and stderr, kills the process on timeout, and resolves a structured object containing success state, output, exit code, elapsed time, and timeout status. Without this wrapper, every compiler and Git call would duplicate risky process code.

terminalLine, processTerminalText, replacePathReferences, and processTerminalTextWithPaths

These helpers turn raw process results into readable terminal output. They combine stdout and stderr, include elapsed time and status, and replace long local paths with shorter display names. They do not compile anything themselves; they make compiler and Git results understandable in the builder UI.

Compile workspace and build orchestration

This is the backend IDE layer. It turns editor state into real files, runs the correct language-specific toolchain, caches outputs, and returns terminal and waveform data.

saveCompileSource({ projectId, fileId, title, fileName, language, role, code, stdin })

saveCompileSource writes the active source file and a metadata file under compileRoot. The object parameter is the frontend's description of the active file. projectId chooses the project workspace, fileId chooses the file folder, fileName chooses the source filename, language chooses the compile profile, role distinguishes HDL design/testbench files, code is the source text, and stdin is optional runtime input. The returned saved object tells the frontend exactly where the backend stored the source.

compileWorkspaceFilesFromPayload(payload, activeFileName, activeLanguage)

This function normalizes the list of files sent by the frontend. It makes sure the active file is included, gives every file a safe id and filename, normalizes language and role, and removes duplicates. It matters because a project workspace is not just one file: C/C++ headers, Java classes, and HDL testbenches may all be needed together.

writeCompileWorkspaceSources(files, targetDir, options)

This async function writes a normalized list of workspace files into a run or cache directory. It can filter by languages or extensions, and it returns written source records that include sourcePath and uniqueName. Many compilers need real files on disk rather than strings in memory, so this function is the bridge between editor state and compiler input.

compileAndRunCode(payload)

compileAndRunCode is the central build engine. Its payload contains language, fileName, code, action, projectId, stdin, workspaceFiles, and forceRebuild. It detects the language, saves the active source, prepares run folders, selects the action, finds tools, runs language-specific compile/run/simulate logic, and returns ok,

language, saved metadata, terminal text, and waveform data when available. It is async because every meaningful operation may touch disk or external processes. Without it, the Compile Code UI would be only a text editor with no real execution path.

installCompilerTools(language)

installCompilerTools looks up the selected language profile and runs Winget packages when automatic installation is configured. Its input is the language id. Its output is a terminal-style result describing whether installation commands succeeded. It exists to reduce manual setup, but it intentionally returns a clear message for languages that do not have automatic installation configured.

C, C++, Java, and HDL build details

These helpers customize compileAndRunCode for languages that need multiple files, binary artifacts, cached builds, testbenches, or waveform output.

cFamilyCompileProfile, cFamilyWorkspaceSources, cFamilyBinaryName, and cFamilyRunOutput

These functions define how C and C++ behave. cFamilyCompileProfile chooses compiler flags, standard version, and whether a file is a header-only syntax check. cFamilyWorkspaceSources selects relevant .c/.cpp/.h/.hpp files. cFamilyBinaryName gives a safe executable name. cFamilyRunOutput formats runtime output. Together they let Compile Code distinguish syntax checking, building a workspace, and running a binary.

javaMainClassName(code, fileName)

Java execution needs a class name, not only a filename. This function reads the code to find a public class first, then any class, then falls back to the filename. compileAndRunCode uses that class name to know which .class file to expect and what to pass to the java runtime.

hdlFilesFromPayload, inferCompileFileRole, normalizeCompileFileRole, hdlModuleNames, and hdlHasWaveDump

These helpers make Verilog/SystemVerilog simulation sane. They identify design files versus testbenches, normalize the role labels, extract module names so a top module can be selected, and check whether a testbench contains \$dumpfile and \$dumpvars. Simulation needs stimulus and waveform dumping; these functions enforce that idea before vvp runs.

parseVcdScopeText(text, source), readHdlWaveform(cacheDir), and clearHdlWaveforms(cacheDir)

parseVcdScopeText converts a VCD waveform file into structured signal data: timescale, max time, signal names, and value changes. readHdlWaveform finds a generated VCD file in the cache directory and parses it. clearHdlWaveforms removes stale VCD files before a fresh simulation run. These functions are why the builder can show a signal scope instead of only text output.

GitHub, public source, and AI context enrichment

These functions let the assistant answer from project evidence, public GitHub links, public profile links, and fetched text without pretending to see private data.

sourceUrlAllowed, parseGitHubSourceUrl, fetchGitHubJson, and fetchLimitedText

These helpers control external reads. sourceUrlAllowed rejects unsafe or unsupported URLs. parseGitHubSourceUrl understands GitHub profile and repository URLs. fetchGitHubJson calls GitHub's public API with appropriate headers. fetchLimitedText retrieves bounded text so a large public file cannot flood the AI context. Together they keep enrichment useful and limited.

fetchGitHubProfileSource, fetchGitHubRepositorySource, and fetchGitHubSourceText

These functions are the GitHub-specific knowledge collectors. A profile source can yield profile metadata and repositories. A repository source can yield README text, repository metadata, and selected source files scored

against the question. The selected snippets become `sourceExcerpts` that the AI can quote or explain. They exist so a question about code can reach public GitHub evidence rather than only project summaries.

`enrichPortfolioContext(context)`

This function receives the context assembled by the frontend and expands it with fetched source excerpts when allowed. It reads `sourceFetches`, fetches safe public text, and returns a richer context object. The function is async because it may call GitHub, fetch same-site files, or read local files. Without it, the AI backend would only see the compact catalog context and would be weaker on uploaded files or public repositories.

Publishing target, authorization cache, and site push

This group is the site-deployment guardrail. It validates the target, verifies access, caches successful authorization, imports from target, and applies generated site files.

`validatePublishRemoteUrl`, `parseGitHubRemote`, `validateCredentialPair`, and `validateCustomDomain`

These functions validate user-provided publishing inputs. repository URLs must be plausible Git remotes, credentials must be paired when supplied, and custom domains must be simple valid domain names. They exist so `configure/authenticate` actions fail early with clear messages instead of half-writing a broken target.

`resolveInsideRoot`, `resolveInsidePortfolioRoot`, `resolveInsideCompileRoot`, and `samePath`

These path helpers defend the workspace boundaries. They turn user-controlled segments into paths inside the intended root and prevent path traversal. They are boring by design, but without them uploads, compile files, and publish sync could write outside the intended folders.

`configurePublishTarget(options)` and `authenticateGitHubForTarget(options)`

`configurePublishTarget` sets up the publish repository, remote, branch, optional custom domain, and stored credentials. `authenticateGitHubForTarget` goes further: it temporarily applies the target, checks Git availability, stores credentials if provided, synchronizes the remote branch, verifies write access, writes the auth cache, and only then keeps the target. This distinction matters: target setup should not be considered complete until write access is proven.

`publishAuthCacheScope`, `writePublishAuthCache`, `publishAuthCachelsFresh`, and `assertPublishAccess`

These functions control the 'do not ask every time' behavior. The cache stores remote, branch, repository, machine/user scope, checked time, expiration, and extended trust history. `assertPublishAccess` first validates repository shape, then reuses a fresh cache if the exact target and machine scope match; otherwise it performs a new write check. Without this group, the app would either annoy the owner constantly or trust stale authorization too broadly.

`syncFromPublishTarget()`

`syncFromPublishTarget` authenticates, clones the selected branch into a temporary folder, checks for a compatible `projects.json`, backs up local compatible files, copies imported site assets into the builder workspace, updates the local draft, and syncs publish files. This is the 'load from target' behavior for moving to another machine or restoring from the live repository.

`syncPortfolioPublishFiles(options)` and `publishSiteChanges(publishAccess)`

`syncPortfolioPublishFiles` copies only allowed `publishPaths` from the builder workspace to the publish mirror. `publishSiteChanges` then synchronizes the branch, bumps asset versions, stages allowed paths, commits changes when needed, and pushes. This pair turns local saved portfolio output into the public website while keeping builder-only files out of the publish target.

Updates, security reporting, AI, and API routing

These functions make the backend responsible for application maintenance and the public assistant endpoint used during local preview.

getUpdateInfo(), getBuilderReleaseDownloadReport(), safeUpdateFileSegment(), powershellSingleQuoted(), and downloadAndLaunchAppUpdate()

getUpdateInfo reads release information and decides whether a newer builder exists.

downloadAndLaunchAppUpdate downloads the latest installer, writes a PowerShell handoff script, closes running builder processes, runs the installer, and relaunches the app. The helper functions sanitize version/file fragments and PowerShell strings so generated scripts are safer. This group exists because a running desktop app cannot replace itself like a normal webpage refresh.

getSecurityReport()

getSecurityReport summarizes what the builder can and cannot know about visitors, downloads, public site headers, local-only protections, and authentication caching. It is intentionally honest: a static GitHub Pages site cannot identify every visitor without an analytics backend.

callOllamaPortfolioAi, extractOpenAiText, extractOllamaText, ruleBasedConversationAnswer, and handlePortfolioAi

These functions make the local AI endpoint. handlePortfolioAi reads the question, conversation, intent, and context; enriches the context; chooses OpenAI if an API key exists; otherwise tries Ollama; otherwise returns a clear unavailable response. The extractor helpers normalize provider response shapes.

ruleBasedConversationAnswer covers simple local fallback conversation. This keeps AI secrets on the backend side and gives the frontend one endpoint to call.

handleApi(request, response, url)

handleApi is the router for every backend endpoint. It handles read endpoints such as catalog, templates, system check, update info, security report, and compiler tools. It rejects non-local write requests, then handles write operations such as AI, code beautify, code save, code compile, tool install, Git authentication, load from target, save draft, apply to site, and upload. It returns true when it handled a route so the static file server does not also try to serve it.

Curated File Guide: script.js

The public website runtime that renders the portfolio, search, project windows, browser history, rich content, links, and AI chat behavior.

Before The Code: File Overview

Abstract

script.js is the public website runtime. It is the JavaScript that turns the static portfolio files into an interactive recruiter-facing site. The HTML page provides stable containers and the JSON catalog provides saved content, but script.js decides what appears, how projects are filtered, how rich text renders, how files become links, how project sections open into full-window views, how browser Back and Refresh interact with those views, how search behaves like a real search tool, and how the AI chat gathers useful context before asking the backend.

The important distinction is that script.js is public and read-only. It should never expose builder-only editing controls. Its job is to present what has already been saved and parsed by the builder. That is why much of the file is about rendering, normalization, routing, search indexing, assistant context selection, and UI state rather than file writes.

Introduction

When a visitor opens the portfolio, index.html loads styles.css, electronics-search.js, and script.js. script.js immediately grabs the DOM elements it will control: the project grid, search input, filters, profile fields, resume area, contact area, AI assistant, and footer. Then it waits for portfolio data. The data can come from an embedded window.__PORTFOLIO_CATALOG__ object in previews or from projects.json on the published website. Once the catalog is loaded, script.js normalizes profile data, renders the hero and contact information, creates category filters, renders project cards, builds the search index, and restores any deep-linked project section.

This file is the main reason the same portfolio can feel alive without requiring a database-backed website. It uses static files as the source of truth, then builds an application-like experience in the browser. That makes the public website easy to host on GitHub Pages or Cloudflare, while still giving visitors search, project windows, AI questions, and responsive navigation.

Background Information

The file works with a catalog generated by the builder parser. That catalog contains categories, projects, profile fields, site content, field styles, rich text blocks, fun facts, and optional site sections. The functions in script.js assume the catalog is already safe enough to display, but they still escape HTML where user-facing strings are inserted into the DOM. Rich text rendering is intentionally structured because portfolio content can include plain text, links, formulas, images, files, code blocks, captions, and nested project subsections.

The website also has a windowing model. Project cards appear inside categories. A project section button opens a full-screen dialog. Inside that dialog, subsections can be clicked like nested folders. Browser history is updated with the project id, section index, and resource path so the browser Back button can move through the same hierarchy. This makes project content feel more like a navigable portfolio than a long flat webpage.

Search is also broader than a normal text filter. The search index contains page sections, project names, category labels, tools, languages, rich text, files, URLs, captions, uploaded text files where fetchable, and electronics keyword expansions. The goal is that a recruiter can type an electronics phrase and land near the relevant artifact, even if the phrase appears inside a project section or file caption rather than on the top card.

Because the website is static, script.js must do work that a database-backed application might normally ask a server to do. It must decide which categories exist, which projects belong to each category, which sections are empty, which files should render as downloads, which links should open externally, and which project paths should be represented in browser history. The price of static hosting is that the browser runtime has to be disciplined. The benefit is that the public site remains easy to deploy, cache, mirror, and serve from GitHub Pages or Cloudflare without a long-running custom web server.

The file also sits between several other files. index.html gives it DOM anchors. styles.css gives visual meaning to the classes it writes. electronics-search.js can provide expanded electronics vocabulary. projects.json provides the portfolio catalog. The AI backend endpoint gives it remote intelligence when available. If one of those contracts changes, script.js is usually the first file that reveals the mismatch because rendering, search, or chat stops behaving correctly.

Purpose Of The File

The purpose of script.js is to convert portfolio data into a polished public experience. The file decides not only what to render, but also what not to render. Empty sections are hidden. Overview content appears as the first readable content when present. Links that look like websites open as websites rather than downloads. Local files become downloadable resources. Images render inline only when they are meant to be shown. AI links are only displayed when related to the question context.

Without script.js, index.html would be mostly a static shell with placeholders. There would be no dynamic project directory, no generated categories, no rich project windows, no browser-aware section routing, no searchable uploaded file text, no AI context gathering, no responsive project rendering, and no dynamic profile/contact population.

Primary Responsibilities

The first responsibility is state hydration. loadProjectCatalog reads the portfolio catalog, normalizes the data, stores it in module-level variables, and triggers rendering. This is the startup path for the public website.

The second responsibility is rendering. Functions such as renderProfile, renderSiteContent, renderFunFacts, renderSiteSections, projectCard, parsedSection, and parsedNodeContent turn structured catalog objects into actual HTML.

The third responsibility is interaction. The dialog functions create the full-window project section view, the routing functions keep browser history synchronized, the search functions build and display selectable results, and the AI functions manage the chat panel.

The fourth responsibility is interpretation. The assistant functions classify questions, gather relevant portfolio context, select public source links, decide whether a question is general or portfolio-specific, and prepare the request sent to the AI backend. This prevents the assistant from blindly dumping generic links for every question.

The fifth responsibility is restraint. Public visitors should see only finished, parsed, saved portfolio output. The file therefore hides empty content, avoids exposing builder edit controls, limits which sources are shown with AI answers, and uses normalized link behavior so a website link does not become a fake downloadable file. This restraint is what separates the public website from the local builder application.

The sixth responsibility is mobile and desktop continuity. The same project data must read cleanly on an iPhone and on a wide monitor. The responsive helper functions estimate project density and media presence, while the dialog and assistant functions keep windows and chat panels usable without letting one part of the page resize unrelated parts. script.js is therefore not only a renderer; it is also the public interaction policy for different devices.

Important Data Objects And Why They Matter

categories, projects, siteSections, profile, siteContent, funFacts, fieldStyles, and the rich variants of those fields are the hydrated catalog state. They are not hardcoded content. They are populated from projects.json or preview data, which is why the same script can run for Maurice's site and also for someone else's freshly installed builder output.

searchableEntries is the in-memory search index. Every item in it has a title, type, context, text, optional project id, optional section index, optional URL, and normalized text. The search functions score and display these entries.

assistantChatHistory and assistantSourceMap are the AI interface memory. assistantChatHistory keeps recent conversation so follow-up questions make sense. assistantSourceMap maps clickable source buttons in an answer back to portfolio search results so a visitor can jump to the relevant project or file.

currentSearchResults and searchResultLimit are the active search session. They remember what the latest query produced and how many items the user wants to see. Without these values, pressing Enter, selecting a result, or changing the result limit would not know which result list is current.

sectionRouteKey and sectionRouteParams are the browser-history contract for project windows. They keep the public URL connected to the currently opened project section and nested subsection. The values are small, but they are what let Refresh reopen the same project view and Back return to the previous view.

Code Walkthrough

The walkthrough below groups related functions where they form one idea. Small helpers are explained together when that is clearer than pretending each one is a separate chapter. Larger functions receive direct explanations of their parameters, internal objects, side effects, return values, and why they are necessary.

Page state, profile, hero, and fun facts

This group turns catalog-level identity and front-page content into visible website content.

normalize(value), normalizeFunFacts(value), and renderFunFacts()

normalize gives the rest of the file a consistent lowercase comparison string. normalizeFunFacts accepts either an array or newline-separated text, trims it, removes empty lines, and caps the visible list. renderFunFacts decides whether the fun facts callout is hidden, then renders either rich blocks or plain short facts. Without this group, top-page personal statements would be inconsistent and could show empty containers.

normalizeSiteContent, normalizeProfile, renderSiteContent, and renderProfile

These functions are the identity hydrators. normalizeSiteContent and normalizeProfile merge catalog data with safe defaults. renderSiteContent updates the hero eyebrow, title, copy, and brand text. renderProfile updates the contact band, links, email/phone behavior, resume section, hero image, brand image, profile photo, and footer owner. These functions make the same HTML shell become a personalized portfolio.

mailComposeLink(email) and phoneLink(phone)

These two helpers convert contact values into browser actions. mailComposeLink creates a Gmail compose URL with the recipient filled in. phoneLink creates a tel: URL so phones and desktop calling apps can offer a call. They are separate functions because email and phone values must be encoded differently.

Dialog dragging, resizing, and window feel

These functions manage draggable section dialogs on the public site. They turn a modal into a controlled full-window or movable document-style view.

clampSectionDialogPosition, anchorSectionDialog, and anchorSectionDialogForResize

clampSectionDialogPosition keeps a dialog within the viewport. anchorSectionDialog converts the browser's current layout position into explicit left/top coordinates so dragging can begin smoothly. anchorSectionDialogForResize does the same before resizing. Their inputs are coordinates and the dialog element. Their side effect is CSS positioning.

beginSectionDialogDrag, moveSectionDialogDrag, endSectionDialogDrag, beginSectionDialogResize, moveSectionDialogResize, and endSectionDialogResize

These functions form pointer-state machines. The begin functions remember the starting mouse/touch point and dialog rectangle. The move functions calculate deltas and update style. The end functions clear active state. They are separate because browser pointer events arrive over time, so the file must remember what operation is active between events.

updateSectionDialogMinimize, toggleSectionDialogMinimized, and enableSectionDialogDrag

These functions keep the dialog's minimized state and interaction affordances in sync. The minimized state changes classes and visible behavior; enableSectionDialogDrag wires the required event handlers once. Without them, the section window would feel like a static modal instead of a controllable portfolio window.

Rich content, links, math, code, and images

This group renders the mixed content created by the builder: text, formulas, links, images, captions, file downloads, and syntax-highlighted code.

escapeHtml, renderInlineMath, renderMultilineInlineText, sanitizeRichInlineHtml, and linkifyRichTextNodes

These functions protect and enrich displayed text. `escapeHtml` prevents raw text from becoming HTML. `renderInlineMath` recognizes formula-like fragments and renders them cleanly. `renderMultilineInlineText` preserves intentional line breaks. `sanitizeRichInlineHtml` removes unsafe tags while keeping allowed inline formatting. `linkifyRichTextNodes` turns recognized URLs into links. Together they keep rich text useful without trusting arbitrary pasted HTML.

normalizeLinkTarget, looksLikeBareWebAddress, looksLikeWebOrContactText, isWebsiteLinkItem, linkAttributes, downloadAttribute, and resourceLink

These functions decide whether a string is a web link, contact link, local download, or empty planned asset. `normalizeLinkTarget` is the main converter; `linkAttributes` adds target and rel for real web links; `downloadAttribute` adds download only for local file assets; `resourceLink` builds the final anchor or muted placeholder. This is the group that prevents LinkedIn or GitHub links from being treated like files.

normalizeCodeLanguage, detectCodeLanguage, tokenizedCodeHtml, and renderRichCodeBlock

These functions make saved code look like code instead of ordinary paragraph text. `detectCodeLanguage` uses aliases and source clues. `tokenizedCodeHtml` highlights common language tokens. `renderRichCodeBlock` wraps the result in a presentable block with language metadata. The functions do not compile code; they render code evidence for public viewing.

richImageCropStyle, richImageDownloadLink, cleanRichImageTitle, renderRichContent, and renderRichFieldContent

These functions render rich editor blocks from the builder. Images may have crop settings, captions, downloadable behavior, or visible inline behavior. `renderRichContent` loops through saved blocks and dispatches to text, image, formula, code, or file rendering. `renderRichFieldContent` applies the same idea to profile/front-page fields. This is where builder-created rich content becomes portfolio-visible content.

Project templates, categories, and responsive project cards

This group maps saved project appearance and project data into project cards that adapt to the visitor's device.

canonicalTemplateId, projectTemplateId, projectTemplateClass, projectTemplateStyle, and applyProjectTemplateToElement

These functions treat templates as appearance choices, not content templates. `canonicalTemplateId` handles legacy template names. `projectTemplateId` extracts the active appearance. `projectTemplateClass` and `projectTemplateStyle` turn that choice into CSS classes and variables. `applyProjectTemplateToElement` applies those variables to project cards or section windows. This keeps project visuals consistent between card view and opened section view.

hydrateProjectCategories, normalizeCategory, setActiveFilter, bindFilterButtons, renderCategoryFilters, categorySection, and projectCard

These functions build the project directory. `hydrateProjectCategories` combines saved categories with projects so empty and custom categories behave correctly. `renderCategoryFilters` creates filter buttons. `categorySection` renders each category band. `projectCard` renders either the modern parsed project model or the older legacy project model. The result is a page organized by categories without hardcoding category names.

inferResponsiveProfile and projectResponsiveProfile

These functions read the project structure and estimate whether the project is simple, balanced, or dense. They count visible sections, nested depth, media presence, and child nodes. The returned profile gives CSS guidance such as content width, card layout, media max width, and touch targets. This is why the website can render projects differently on phones and PCs without Codex manually rearranging each project.

Search engine behavior

This group turns portfolio content and project evidence into a local searchable index with scoring, highlighting, and result navigation.

flattenSearchText, richTextTerms, addSearchEntry, searchSnippet, entryScore, and searchResultsFor

These functions define a search entry and how it ranks. `flattenSearchText` collapses arrays, objects, and strings into searchable text. `richTextTerms` extracts text-like values from rich blocks. `addSearchEntry` stores normalized text. `entryScore` rewards exact title matches, starts-with matches, context hits, type hits, word hits, and useful kinds such as files or sections. `searchResultsFor` scores, sorts, and de-duplicates entries. The user experiences this as a real search dropdown rather than a simple project-name filter.

filesBrowserSearchable and indexSearchableFileText

`filesBrowserSearchable` decides which uploaded or linked files can be fetched as text in the browser. `indexSearchableFileText` fetches those files, appends a bounded amount of file text to the entry, and refreshes the dropdown. This allows code, logs, CSV, Markdown, Verilog, and similar files to become searchable by content when the browser can access them.

addProjectSearchEntries, addSiteSectionSearchEntries, rebuildSearchIndex, renderSearchResults, updateSearchDropdown, and goToSearchResult

These functions build and use the full index. Projects contribute titles, categories, tools, languages, sections, files, rich text, and electronics vocabulary. Site sections contribute profile/resume/contact/process content. `renderSearchResults` shows the selectable list; `updateSearchDropdown` highlights visible text; `goToSearchResult` scrolls, opens a section window, or opens a link depending on result kind.

Portfolio AI context and conversation

This group decides what the public AI assistant should know for a question and how the chat should behave.

assistantQuestionIntent and its classifier helpers

`assistantQuestionIntent` uses helpers such as `assistantQuestionIsCasual`, `assistantQuestionHasPortfolioIntent`, `assistantQuestionLooksConceptual`, and `assistantQuestionIsEngineeringRelated`. The output can be `general_conversation`, `general_engineering`, `general_knowledge`, or `portfolio_specific`. That intent decides whether the assistant should answer generally or gather portfolio context first. This prevents a question like 'what is embedded systems' from immediately dumping Maurice's project links.

assistantContextForQuestion, assistantProjectEvidence, assistantPublicProfileLinks, assistantSourcesForDisplay

These functions assemble the request context. They identify named projects, collect matching search results, gather public profile links, include project evidence, and choose which source buttons should display. `assistantSourcesForDisplay` is deliberately strict so the assistant does not append random generic links.

assistantGeneralEngineeringAnswer and assistantLocalAnswer

These provide local fallback intelligence. `assistantGeneralEngineeringAnswer` contains direct explanations for embedded systems, op amps, VCO/PWM, FPGA/ASIC, STM32, PCB, and power topics. `assistantLocalAnswer` handles greetings, identity questions, catalog-loading cases, no-match cases, and summary answers from local search results. This keeps the assistant conversational even when the backend AI is unavailable.

askRemoteAssistant and answerAssistantQuestion

`askRemoteAssistant` sends question, context, conversation, intent, and web-search permission to the backend endpoint with a timeout. `answerAssistantQuestion` is the UI orchestrator: it appends the user message, remembers conversation, shows a pending assistant message, decides intent, gathers sources, calls the remote

assistant, falls back locally if needed, replaces the pending answer, remembers the assistant response, and restores input focus. It is async because network AI calls and source fetches take time.

appendAssistantMessage, replaceAssistantMessage, renderAssistantAnswerContent, clearAssistantChat, and updateAssistantPanelGrowth

These functions are the chat interface. renderAssistantAnswerContent converts ChatGPT-like plain text into paragraphs, bullets, and code blocks. append/replace message functions update the DOM. clearAssistantChat resets history and source maps. updateAssistantPanelGrowth grows the AI console upward on desktop without resizing the contact panel. This group makes the assistant feel like a clean chatbot instead of a static Q&A box.

Parsed project windows and browser routing

This group turns nested project sections into clickable windows and makes browser Back/Forward participate.

nodeHasRenderableContent, sectionHasRenderableContent, nodeOverviewDetails, parsedNodeContent, parsedSectionContent, parsedSection, and projectCard

These functions decide what exists on the public site. Empty nodes are not rendered. Overview content is shown automatically and collapsibly when it exists. Child sections become clickable cards. Files become inline file links. projectCard wraps all of that into the category card. This group enforces the rule that only valid created content appears.

pathToString, pathFromString, nodeAtPath, nodeChildren, sectionRouteState, sectionRouteUrl, sectionRouteFromLocation, and sameSectionRoute

These are routing helpers for nested project windows. A path such as 0.2.1 means 'walk through child indexes to reach a subsection.' The route functions translate project id, section index, and resource path into URL/search/history state and back. They are necessary because the browser Back button must know which project window is currently open.

ensureSectionDialog, openParsedSection, closeSectionDialog, closeOrStepBackSectionDialog, applySectionRoute, and restoreSectionRouteAfterCatalogLoad

ensureSectionDialog creates the reusable dialog. openParsedSection finds the project/section/node, applies the project appearance template, updates route state, writes the title, renders content, and opens the full-window dialog. close/step-back functions decide whether to go up one nested level or close to the main page. applySectionRoute and restoreSectionRouteAfterCatalogLoad connect this behavior to browser history and page refresh.

loadProjectCatalog

loadProjectCatalog is the startup pipeline. It loads either embedded preview data or projects.json, stores catalog pieces in state, renders profile/site/facts/sections/filters/projects, rebuilds search, updates the search dropdown, and restores deep-linked section windows. Without this function, the static HTML shell would never become the actual portfolio.

Curated File Guide: index.html

The public portfolio shell: metadata, semantic page structure, runtime hooks, visible sections, and script/style entry points.

Before The Code: File Overview

Abstract

index.html is the public website shell. It is the first file a recruiter or visitor receives from the static host. It defines the document metadata, search-engine hints, social-card metadata, favicon and app icons, structured data, visible page sections, semantic landmarks, and the fixed DOM ids that script.js later fills with real portfolio content. The file is intentionally light on final personal content because the builder writes most custom data into projects.json and script.js renders it dynamically.

The most important idea is that index.html is an interface contract. It tells the browser, search engines, social media previews, accessibility tools, styles.css, and script.js where things are and what each major region means. If script.js is the behavior layer, index.html is the skeleton that gives that behavior safe attachment points.

Introduction

The document begins with standard HTML setup: language, character set, viewport, description, robots instructions, author, and the AI endpoint metadata. These are not decorative tags. They affect how the page is indexed, how it scales on phones, how icons appear, and where the AI chat sends requests. After the metadata, index.html defines the visible page from top to bottom: header, hero, fun facts, builder download, summary metrics, projects, resume, dynamic sections, process, contact/AI, and footer.

Most of the actual portfolio content is not typed directly in this file. That is deliberate. The builder is supposed to let users change names, images, resumes, project categories, links, and rich text without editing HTML manually. Therefore index.html supplies stable placeholders such as #hero-title, #project-grid, #resume, #dynamic-sections, #contact, and #ask-ai. script.js finds those placeholders and fills them from the catalog.

Background Information

A static portfolio can be hosted cheaply and reliably if the page can be built from HTML, CSS, JavaScript, and JSON. index.html is designed for that model. It avoids server-only assumptions and includes assets with relative paths. It loads electronics-search.js before script.js because script.js can use the electronics search vocabulary when building project search terms.

The file also contains schema.org JSON-LD. That structured data gives search engines a machine-readable hint that the page is a ProfilePage about a Person. The builder later updates the visible content, but this baseline makes the page understandable even before dynamic content fully loads.

The page uses semantic sections and labels because public portfolio viewers include recruiters, mobile users, browser accessibility tools, and search crawlers. A good static shell gives all of them landmarks. For example, the projects section is labeled by projects-title, the resume section is hidden until a resume URL exists, and the AI assistant has a labelled panel so it can be reached from navigation and screen readers.

The file is intentionally not a finished personal biography. It is a reusable public shell. A fresh install should not hardcode Maurice-specific content, while Maurice's own built portfolio can still publish his profile through the catalog. That is why placeholder labels such as Portfolio Owner, Portfolio, and generic engineering copy exist here. The builder-generated catalog is the layer that specializes the site.

The HTML also controls load order. CSS loads before the page renders so layout and colors are available immediately. `electronics-search.js` loads before `script.js` because `script.js` may call the electronics keyword helper while building the search index. `script.js` loads at the end of the body so the DOM elements already exist by the time it queries them. If `script.js` loaded too early without deferring, many `document.querySelector` calls would return null.

Purpose Of The File

The purpose of `index.html` is to guarantee that the public portfolio has a stable starting structure. Without it, `script.js` would have nowhere reliable to render projects, no header to navigate from, no AI panel to attach events to, no resume viewer to populate, no contact band to update, and no metadata to help browsers or search engines understand the page.

It also separates permanent page structure from changeable portfolio content. The structure changes slowly; the content changes whenever the builder saves and publishes. That separation is what allows the builder to generate portfolio updates without rewriting the entire website architecture each time.

The file is also the public accessibility foundation. Section headings, labels, aria attributes, hidden states, object fallback text, and semantic main/header/footer landmarks make the site more understandable to assistive technology. `script.js` can only enhance that foundation; it cannot rescue a shell that has no stable names or landmarks.

Another purpose is failure tolerance. If `projects.json` fails to load, the page should still show a header, hero, search area, process section, and footer. If the resume is not configured, the resume section remains hidden. If the profile image is missing, the image elements stay hidden. This is what keeps a partially configured portfolio from looking broken.

Primary Responsibilities

The first responsibility is document identity. The head contains title, description, canonical URL, social tags, icons, AI endpoint, and structured data. These values determine how the page appears to crawlers, shared links, browser tabs, and mobile install surfaces.

The second responsibility is visible layout landmarks. The body defines the header, main content regions, sections, and footer. `styles.css` controls how they look, and `script.js` controls most dynamic filling.

The third responsibility is runtime hooks. Every id and class that `script.js` queries is part of a contract. Removing `#project-grid`, `#project-search`, `#ask-ai`, `#contact`, `#resume`, or `#dynamic-sections` would break the corresponding rendering logic.

The fourth responsibility is graceful fallback. The page contains placeholder text and hidden sections so the website has a sensible shape even before `projects.json` finishes loading. If a profile image or resume does not exist, those areas remain hidden instead of showing broken controls.

The fifth responsibility is hosting compatibility. The document uses ordinary relative paths and static assets. There is no server-side template language, no database call, and no build step required at request time. That is why the same page can be served from GitHub Pages, Cloudflare, or a local preview server.

The sixth responsibility is public trust. The page exposes recruiter-facing areas only: projects, resume, process, contact, download builder, and AI assistant. It does not include local builder controls, Git publishing buttons, compiler controls, or editing widgets. That separation protects the private builder workflow from public visitors.

Important Elements And Why They Matter

The header is the main navigation surface. Its brand link returns to the top, the nav links jump to AI, projects, resume, process, and contact, and the avatar link can take the visitor to contact information. The brand image and text are later customized by script.js.

The hero section is the first impression. It contains a hidden hero image, overlay shade, eyebrow, title, copy, project/resume buttons, GitHub button, and Ask AI jump link. These are all visible near the top because recruiter-facing content should not require deep scrolling.

The project section is the core portfolio directory. It includes a search input, filter controls, and the empty project grid that script.js fills. The search input is deliberately placed in the section heading because search is a primary navigation method, not an afterthought.

The contact band is last. It holds contact information and the AI assistant panel. The AI lives in this area because it is a helper for exploring the portfolio, while the contact section remains the final page region.

The hidden attributes are part of the contract. They do not mean the feature is dead; they mean script.js will reveal the feature only when the catalog contains the required data. This prevents empty resume viewers, empty avatars, and profile images without sources from appearing to visitors.

The IDs are also part of the contract. #project-grid, #project-search, #project-filters, #resume, #dynamic-sections, #contact, #ask-ai, #ai-assistant-form, #ai-assistant-input, and #ai-assistant-log are the anchors script.js expects. Renaming them would not only change HTML; it would break runtime behavior.

Code Walkthrough

The walkthrough below groups related functions where they form one idea. Small helpers are explained together when that is clearer than pretending each one is a separate chapter. Larger functions receive direct explanations of their parameters, internal objects, side effects, return values, and why they are necessary.

Head metadata and search identity

The head makes the page understandable before JavaScript renders anything.

charset, viewport, description, robots, author, and canonical link

The charset tells the browser to interpret the file as UTF-8. The viewport makes the page scale correctly on phones. The description gives search engines a concise page summary. robots tells crawlers that indexing is allowed. author is a generic placeholder updated by published content or metadata later. The canonical link is empty in the template because the final domain can vary.

portfolio-ai-endpoint meta tag

This meta tag gives script.js a backend URL for the portfolio AI assistant. assistantEndpoint reads it before falling back to /api/portfolio-ai. Keeping the endpoint in metadata lets the same JavaScript bundle run locally and publicly while pointing to different AI backends.

Open Graph and Twitter card tags

These tags control how the portfolio looks when shared on social platforms or messaging apps. The defaults are generic because the builder can later fill real owner, title, image, and description values.

favicon and touch-icon links

These links tell browsers and mobile operating systems which icons to show in tabs, bookmarks, install prompts, and home-screen shortcuts. They point at the OMB asset set so the portfolio has a recognizable identity.

ProfilePage JSON-LD script

This structured data gives search engines a machine-readable summary of the page as a profile page about a person. The values are conservative placeholders. The important part is the shape: Person, name, image, URL, sameAs, and knowsAbout. It gives crawlers context even though the visible profile is populated dynamically.

Header and navigation contract

The header gives visitors immediate navigation and gives script.js stable places to insert brand/profile visuals.

header#top and brand link

The header has id top so Back to top links can target it. The brand anchor points to #top and contains the brand image, OMB engraving, title, and subtitle. script.js later replaces brand text and image from profile data. If these classes disappeared, renderProfile would lose its brand hooks.

site-nav links

The navigation links jump to Ask AI, Projects, Resume, Process, and Contact. These anchors match section ids later in the file. They are simple hash links so the page works on static hosting and does not need a router.

header-avatar

The avatar link starts hidden because not every portfolio has a profile image. renderProfile reveals it and sets the image when profile data provides one. Its href points to contact so the image acts as a fast route to contact information.

Hero, fun facts, download, and metrics

These are top-of-page sections that help a visitor orient quickly.

hero section

The hero contains a hidden image, shade overlay, eyebrow, h1 title, copy, action buttons, optional GitHub button, and Ask AI jump link. script.js updates the text and images. The section is labelled by hero-title, which improves accessibility and gives the document a clear main heading.

fun-facts-callout

This hidden section becomes visible only when the catalog includes fun facts or rich fun fact blocks. It sits near the top so personality appears before deep project scrolling.

builder-download-section

This section exposes the latest Windows installer link. It is public because visitors may want to clone/install the builder app. The href points to the stable latest release asset. The download attribute suggests download behavior, but the browser and GitHub ultimately decide how the file is handled.

summary-band metrics

The metrics display project count, category count, and artifact promise. script.js updates the first two after loading projects.json. The third is static because the builder portfolio consistently supports source, diagrams, and reports.

Projects, resume, dynamic sections, process, contact, and AI

These sections are the public portfolio body. Most content is populated dynamically.

projects section

This is the main project directory. It has a heading, search input, filter bar, and empty project-grid. script.js builds filter buttons, project categories, project cards, parsed section buttons, search results, and nested section windows from this area.

resume section

The resume section starts hidden because a fresh portfolio may not have a resume. renderProfile reveals it, sets the PDF object data, and updates Open/Download links when profile.resumeUrl exists. The object element gives an in-page PDF viewer when the browser supports it.

dynamic-sections container

This div is where user-created main portfolio sections render. The builder can add personal life, professional profile, social links, hobbies, or other lighter sections without editing index.html.

process section

The process section is a stable explanatory band describing problem/constraints, build artifacts, and results/reflection. It gives the page a professional engineering narrative even before specific custom sections are added.

contact-band and ask-ai panel

The contact band is intentionally last. It holds profile photo, contact details, external links, and the AI assistant. The AI form, log, input, status, and clear button have fixed ids because script.js attaches the chatbot behavior directly to them.

footer and script loading

The footer provides Back to top and copyright year. The scripts load electronics-search.js before script.js so the search vocabulary is available when script.js builds the index. Version query strings help browsers fetch updated CSS and JS after publishing.